MODULE 02 | FILES & DIRECTORIES

Files & Directories

Copy, move, delete, search and read files like a pro, without breaking anything.

Mission	Topic	Commands / Concepts
1	Copy and Move: cp and mv	cp, mv, rename
2	Remove Safely: rm and Friends	rm, rm -r, rm -i, rmdir
3	Find Things: find and Wildcards	find, -name, -type, -size, -mtime, -exec
4	Read Files: cat, less, head, tail	cat, less, head, tail, tail -f, nl, tac
5	Inspect and Link: file, stat, du, df, ln	file, stat, du, df, ln, ln -s
Boss	Cleanup Crew	Everything in this module

How to use this guide: read each mission, type every example yourself in the Tux Tutor terminal, attempt the practice tasks, then check the answers at the end. Use a throwaway practice folder so mistakes cost nothing.

Module Overview

What you will learn

- Copy and move files and folders (`cp`, `mv`) and use `mv` to rename.
- Delete safely (`rm`) and understand why there is no Recycle Bin.
- Search the filesystem by name, type, size and age (`find`).
- Read files in full or in part (`cat`, `less`, `head`, `tail`).
- Inspect files and disk usage (`file`, `stat`, `du`, `df`) and create links (`ln`).

Setup: build a practice area

Run this once so every example in the guide works as shown.

```
$ mkdir -p ~/lab/{docs,src,backup}
$ cd ~/lab
$ touch docs/{a,b,c}.txt src/main.py src/util.py
$ echo "hello linux" > docs/a.txt
```

Warning
Linux has **no undo** for deletions. Files removed with `rm` do not go to a trash folder. Always double check the path before pressing Enter.

MISSION 1 | +50 XP

Copy and Move: cp and mv

Goal: Duplicate, relocate and rename files and folders.

cp - Copy

```
$ cp a.txt b.txt              # copy file to a new name
$ cp a.txt docs/              # copy into a folder (keeps name)
$ cp a.txt b.txt c.txt backup/ # copy several files into backup/
$ cp -r docs docs_copy        # -r copies a folder and its contents
$ cp -i a.txt b.txt           # ask before overwriting
$ cp -v a.txt b.txt           # verbose: show what happened
$ cp -p a.txt b.txt           # preserve time and permissions
```

Option	Meaning
`-r` / `-R`	Recursive. **Required** for directories.
`-i`	Interactive: prompt before overwrite.
`-n`	Never overwrite an existing file.
`-u`	Copy only if source is newer.
`-a`	Archive: recursive and preserves everything (best for backups).

mv - Move and Rename

`mv` moves a file to another place. If the destination is in the same folder, it simply **renames** it. No `-r` is needed for folders.

```
$ mv old.txt new.txt          # rename
$ mv new.txt docs/            # move into docs
$ mv src/*.py backup/         # move all .py files
$ mv -i a.txt docs/           # ask before overwriting
$ mv oldfolder newfolder      # rename a folder
```

Trailing slash
`cp file dir/` copies into the directory. If `dir` does not exist, `cp file dir` creates a **file** called dir. Add the slash to be safe: if the folder is missing you get an error instead of a surprise.

Bulk rename idea

```
$ for f in *.txt; do mv "$f" "old_$f"; done   # add a prefix to every .txt
```

Practice

- Copy `docs/a.txt` to `backup/a.bak` using one command.
- Copy the whole `docs` folder to `docs_v2`, then rename it to `docs_final`.
- Move all `.py` files from `src` to `backup` with a wildcard.

MISSION 2 | +50 XP

Remove Safely: rm and Friends

Goal: Delete files and folders without deleting the wrong thing.

rm - Remove

```
$ rm a.txt                  # delete a file
$ rm a.txt b.txt            # delete several
$ rm *.log                  # delete all .log files
$ rm -i notes.txt           # ask for confirmation
$ rm -r old_folder          # delete a folder and everything inside
$ rm -rf build/             # force: no prompts, ignore missing files
$ rmdir empty_folder        # only works if the folder is empty
```

Option	Meaning
`-i`	Prompt before each deletion.
`-I`	Prompt once if deleting 3+ files or recursively.
`-r`	Recursive: needed for directories.
`-f`	Force: never prompt, ignore errors.
`-v`	Show each file as it is removed.

Warning
Never run `rm -rf /`, `rm -rf /*` or `rm -rf ~`. A stray space can be catastrophic: `rm -rf ~/ lab` deletes your home folder **and** a folder named lab. Modern systems block `rm -rf /`, but not `rm -rf ~`.

Habits that keep you safe

- Run `ls` with the same pattern first: `ls *.log`, then `rm *.log`.
- Use `rm -i` or `rm -I` while learning.
- Double check pwd before deleting with relative paths.
- Prefer `rm -r folder` over `rm -rf folder` unless you really need force.
- Do not run `sudo rm -rf` unless you are certain of every character.

Dealing with odd names

```
$ rm -- -weird.txt          # file whose name starts with a dash
$ rm "my file.txt"          # name containing a space
```

Practice

- Create `trash/` with five `.tmp` files, preview them with `ls *.tmp`, then delete them.
- Delete the folder `docs_final` with a recursive command and verbose output.
- What happens if you run `rmdir src` while it contains files?

MISSION 3 | +50 XP

Find Things: find and Wildcards

Goal: Locate files by name, type, size or age, and act on the results.

find syntax

```
find  WHERE  CONDITIONS  ACTION

$ find . -name "a.txt"           # exact name, from current folder
$ find ~ -name "*.txt"           # all .txt files under home
$ find . -iname "readme*"        # case-insensitive
$ find . -type f                 # files only
$ find . -type d                 # directories only
```

Test	Meaning
`-name`, `-iname`	Match by filename (quote patterns!).
`-type f|d|l`	File, directory or symlink.
`-size +10M`	Larger than 10 MB (`-1k` = smaller than 1 KB).
`-mtime -7`	Modified in the last 7 days (`+30` = older than 30).
`-user bob`	Owned by bob.
`-perm 644`	Exact permission match.
`-empty`	Empty files or folders.
`-maxdepth 2`	Do not go deeper than 2 levels.

Acting on results

```
$ find . -name "*.tmp" -delete              # delete matches
$ find . -name "*.log" -exec ls -l {} \;     # run a command on each
$ find . -type f -size +100M                  # big files
$ find . -name "*.py" -mtime -2               # edited in last 2 days
$ find /var/log -name "*.log" 2>/dev/null     # hide permission errors
```

Tip
`2>/dev/null` throws away error messages (like 'Permission denied') so they do not clutter output. You will see more about streams in Module 03.

Quick alternatives

- `locate name` is very fast, using a prebuilt index (update with `sudo updatedb`).
- `which cmd` finds where a **program** lives.
- `ls -R | grep name` is a rough search that is easy to remember.

Practice

- Find all `.py` files under `~/lab`.
- Find files larger than 1 MB in your home folder (`2>/dev/null`).

- Find `.txt` files modified in the last day and print their long listing with `-exec ls -l {} \;`.

MISSION 4 | +50 XP

Read Files: cat, less, head, tail

Goal: View whole files, the first or last lines, and live logs.

cat - show the whole file

```
$ cat a.txt
hello linux
$ cat a.txt b.txt         # join and print two files
$ cat -n a.txt            # with line numbers
$ cat > new.txt           # type text, finish with Ctrl+D
$ cat a.txt b.txt > all.txt   # combine files into one
```

less - page through big files

`cat` dumps everything at once. For long files use `less`: Space = next page, b = back, `/word` = search, n = next match, g/G = top/bottom, q = quit.

```
$ less /var/log/syslog
```

head and tail

```
$ head file.txt          # first 10 lines
$ head -n 3 file.txt     # first 3 lines
$ tail file.txt          # last 10 lines
$ tail -n 20 file.txt    # last 20 lines
$ tail -n +5 file.txt    # from line 5 to the end
$ tail -f app.log        # follow: print new lines as they arrive (Ctrl+C to stop)
$ head -n 12 f.txt | tail -n 3   # lines 10 to 12
```

Tip
`tail -f` is the standard way to watch a log while you test an application.

Other readers

Command	Use
`tac file`	Print lines in reverse order.
`nl file`	Number lines.
`wc -l file`	Count lines.
`file x`	Show what type of file x is.

Practice

- Create a 30-line file with `seq 1 30 > nums.txt`, then show lines 11 to 15 using head and tail.
- Use `tail -f` on a file in one terminal while appending to it with >> in another.
- Combine `a.txt`, `b.txt` and `c.txt` into `all.txt` using `cat`.

MISSION 5 | +50 XP

Inspect and Link: file, stat, du, df, ln

Goal: Understand what a file is, how big it is, and how to create shortcuts.

Know your files

```
$ file photo.jpg
photo.jpg: JPEG image data, 1920x1080
$ stat a.txt            # size, permissions, timestamps, inode
$ ls -lh                # human-readable sizes
```

Disk space

```
$ df -h                 # free space per filesystem
$ du -sh docs           # total size of one folder
$ du -h --max-depth=1 .  # size of each subfolder
$ du -sh * | sort -h    # which items are biggest?
```

Command	Answers the question
`df`	How full is the disk?
`du`	What is using the space?

Links

A **symbolic link** (symlink) is a shortcut that points to another path. A **hard link** is a second name for the same data.

```
$ ln -s /var/log/syslog mylog       # symbolic link
$ ls -l mylog
lrwxrwxrwx 1 haani haani 15 Nov 14 mylog -> /var/log/syslog
$ ln a.txt a_hard.txt               # hard link
$ readlink mylog                    # show where a symlink points
```

Symlink vs hard link
Delete the target and a symlink becomes **broken**. A hard link keeps working, because the data lives until its last name is removed. Hard links cannot cross filesystems or point to folders.

Archive basics (bonus)

```
$ tar -czf backup.tar.gz docs/      # create compressed archive
$ tar -tzf backup.tar.gz            # list contents
$ tar -xzf backup.tar.gz            # extract
```

Practice

- Find which three folders in your home take the most space.
- Create a symlink `latest` pointing to `docs/a.txt` and read through it with `cat`.
- Archive `~/lab` into `lab.tar.gz`, list it, then extract it into `/tmp`.

BOSS BATTLE | +150 XP

Boss: Cleanup Crew

Goal: combine everything from this module without hints.

Scenario

A messy project folder has duplicate files, old logs and no backup. You must organise it, back it up, and delete the junk without losing anything important.

Tasks

1. Build the practice area: `~/mess` with files `a.txt`, `b.txt`, `old1.log`, `old2.log`, `report.tmp` and folder `data/`.
2. Create `~/mess/backup/` and copy every `.txt` file into it, preserving names.
3. Rename `report.tmp` to `report.txt` and move it into `data/`.
4. Use `find` to list all `.log` files under `~/mess`.
5. Delete those log files with `find ... -delete` (preview first!).
6. Show the size of `~/mess` in human-readable form.
7. Put the word `done` in `data/status.txt` and display the last line with `tail`.
8. Create a symlink `~/mess/shortcut` pointing to `data/status.txt`.

Sample solution

```
mkdir -p ~/mess/data && cd ~/mess
touch a.txt b.txt old1.log old2.log report.tmp
mkdir backup && cp *.txt backup/
mv report.tmp data/report.txt
find . -name "*.log"
find . -name "*.log" -delete
du -sh .
echo done > data/status.txt && tail -n 1 data/status.txt
ln -s data/status.txt shortcut
```

Check Your Understanding

Q1. Which flag lets `cp` copy a directory?

Q2. How do you rename a file in Linux?

Q3. Does `rmdir` delete folders that contain files?

Q4. What does `find . -name "*.tmp" -delete` do?

Q5. How do you follow a log as it grows?

Q6. Show lines 10 to 12 of a file.

Q7. Difference between `df` and `du`?

Q8. What does `ln -s` create?

Q9. Why quote patterns in find (`-name "*.txt"`)?

Q10. Which flag asks for confirmation before deleting?

Answers

#	Answer
1	-r
2	mv oldname newname
3	No, only empty ones. Use rm -r.
4	Deletes every .tmp file under the current folder.
5	tail -f logfile
6	head -n 12 f | tail -n 3
7	df shows free space per disk; du shows what a folder uses.
8	A symbolic link (shortcut).
9	So the shell does not expand the wildcard before find sees it.
10	rm -i

Quick Reference Cheat Sheet

Command	Purpose	Example
cp -r a b	Copy file or folder	cp -r docs backup/
mv a b	Move or rename	mv old new
rm -ri dir	Delete with prompts	rm -ri tmp/
find . -name	Search by name	find . -name "*.py"
cat / less	Read a file	less big.log
head -n / tail -n	First / last lines	tail -n 20 f
tail -f	Follow a log	tail -f app.log

Command	Purpose	Example
du -sh	Folder size	du -sh *
df -h	Disk space	df -h
ln -s	Symbolic link	ln -s target link
tar -czf	Compress	tar -czf a.tgz dir

Up next: Module 03
Text Processing. Use wc, grep, sed, awk, sort, uniq and cut with pipes to analyse data and logs.